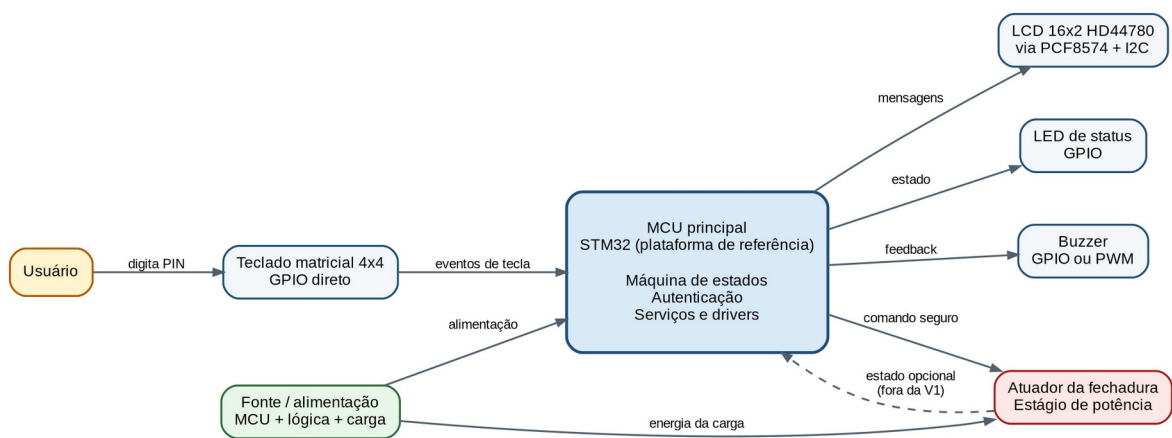


# FECHADURA ELETRÔNICA

Requisitos, arquitetura e plano de implementação - V1



*Baseline técnico para desenvolvimento top-down, incremental e verificável*

Versão do documento: 1.0  
Status: Baseline para início do projeto  
Data: 13 de julho de 2026

# Controle do documento

| Campo                    | Valor                                                            |
|--------------------------|------------------------------------------------------------------|
| Projeto                  | Fechadura eletrônica com STM32, teclado matricial 4x4 e LCD 16x2 |
| Versão do produto        | V1 - protótipo funcional e confiável                             |
| Versão deste documento   | 1.0                                                              |
| Status                   | Baseline de requisitos e arquitetura                             |
| Plataforma de referência | STM32F103C8T6 / Blue Pill para desenvolvimento                   |
| Estratégia de execução   | Superloop cooperativo, sem RTOS                                  |
| Credencial               | PIN numérico fixo de 4 dígitos, compilado no firmware            |
| Interface de usuário     | Teclado 4x4, LCD 16x2, LED e buzzer                              |
| Expansão de I/O          | PCF8574 dedicado ao LCD via I2C                                  |

## Natureza do documento

Este documento é uma baseline de engenharia. Ele fixa o comportamento esperado da V1, define fronteiras entre módulos e estabelece critérios de aceitação. Decisões ainda não tomadas estão registradas explicitamente; não devem ser preenchidas por suposição durante a implementação.

## Sumário

1. Objetivo e princípios de engenharia
2. Escopo da V1
3. Visão do sistema e casos de uso
4. Requisitos funcionais
5. Requisitos não funcionais
6. Máquina de estados e comportamento
7. Arquitetura de hardware
8. Arquitetura de software e regras de dependência
9. Estrutura de pastas
10. Responsabilidades dos módulos
11. Modelo de execução, tempo e concorrência
12. Estratégia de erros, falhas e segurança
13. Estratégia de testes e validação
14. Plano incremental de desenvolvimento
15. Definição de pronto e critérios de release
16. Riscos, decisões pendentes e próximos passos
17. Apêndices: interfaces, rastreabilidade e checklist de revisão

## 1. Objetivo e princípios de engenharia

O objetivo da V1 é entregar uma fechadura eletrônica funcional, determinística e testável, capaz de receber um PIN por teclado, validar a credencial, comandar um atuador por tempo limitado e informar o estado ao usuário. A arquitetura deve ser simples o suficiente para ser compreendida e depurada, mas organizada para permitir troca localizada de componentes sem reescrever a lógica central.

### 1.1 Estratégia de desenvolvimento

- **Top-down para o sistema:** requisitos, casos de uso, estados e interfaces são definidos antes dos drivers.
- **Risk-driven para o hardware:** riscos técnicos como acionamento da carga, temporização do LCD e ruído do teclado devem ser prototipados cedo.

- **Incremental:** cada subsistema é validado isoladamente antes da integração.
- **Não bloqueante por padrão:** delays longos não podem paralisar a aplicação; temporizações são modeladas por deadlines.
- **Dependências unidirecionais:** a aplicação depende de capacidades, não de HAL, registradores ou componentes específicos.
- **Sem abstração especulativa:** uma interface existe quando protege uma variação relevante, melhora testes ou delimita responsabilidade.

### Princípio central

A lógica da fechadura não deve conhecer HAL\_GPIO\_WritePin, HAL\_I2C\_Master\_Transmit, PCF8574, HD44780 ou pinos físicos. Ela deve operar sobre conceitos como tecla, credencial, mensagem, indicação, tempo e comando de destravamento.

## 1.2 Objetivos mensuráveis da V1

- Inicializar em estado travado e seguro após power-on ou reset.
- Aceitar PIN numérico de 4 dígitos pelo teclado 4x4.
- Ocultar os dígitos no LCD, exibindo apenas marcadores de posição.
- Destravar somente após credencial válida e relatar sucesso ao usuário.
- Religar o estado travado automaticamente após tempo configurável.
- Aplicar bloqueio temporário após três tentativas inválidas consecutivas.
- Impedir energização indefinida do atuador, mesmo se a interface de usuário falhar.
- Permitir teste unitário da máquina de estados e autenticação sem hardware real.
- Conter falhas de um driver sem corromper o estado lógico do sistema.

## 2. Escopo da V1

### 2.1 Incluído

- MCU STM32 como controlador principal.
- Teclado matricial 4x4 conectado diretamente a GPIOs do MCU.
- LCD 16x2 compatível com HD44780, em modo de 4 bits, conectado por módulo PCF8574 via I2C.
- PIN fixo de 4 dígitos compilado no firmware.
- Confirmação da credencial pela tecla # e cancelamento/limpeza pela tecla \*.
- LED de status com padrões simples e não bloqueantes.
- Buzzer para feedback sonoro de tecla, sucesso, erro e bloqueio.
- Atuador comandado por estágio de potência com estado seguro e tempo máximo de acionamento.
- Base de tempo em milissegundos e utilitários de deadline/elapsed time.
- Máquina de estados cooperativa, executada em superloop.
- Política de três tentativas inválidas e bloqueio temporário.
- Testes unitários da lógica de aplicação em host e testes de integração no alvo.

### 2.2 Explicitamente fora do escopo

- Alteração de PIN pelo usuário.
- Armazenamento persistente de credenciais em Flash, EEPROM ou banco externo.
- Múltiplos usuários, níveis de acesso ou auditoria de acessos.
- Bluetooth, Wi-Fi, RFID, biometria, aplicativo móvel ou nuvem.
- RTOS, multithreading ou filas de mensagens de kernel.
- Criptografia de credenciais e proteção contra extração física do firmware.
- Sensor de porta aberta/fechada ou confirmação mecânica do trinco.
- Atualização remota de firmware, bootloader customizado e OTA.
- Certificação funcional, segurança física de produto ou conformidade normativa.
- Interface genérica universal para qualquer tipo de dispositivo.

### Limite de segurança

Esta V1 é apropriada como projeto didático e protótipo de engenharia. PIN fixo em firmware, ausência de sensor de posição e ausência de proteção contra ataques físicos impedem tratá-la como produto de segurança real.

## 2.3 Parâmetros iniciais configuráveis

| Parâmetro                | Valor inicial          | Observação                                            |
|--------------------------|------------------------|-------------------------------------------------------|
| Tamanho do PIN           | 4 dígitos              | Fixo na V1; não criar generalidade sem requisito.     |
| PIN de desenvolvimento   | Definido em app_config | Nunca documentar o valor real em repositório público. |
| Timeout de entrada       | 15 s                   | Reiniciado a cada tecla válida.                       |
| Tempo destravado         | 3 s                    | Deve ser menor que o limite térmico da carga.         |
| Tentativas inválidas     | 3                      | Contagem consecutiva; sucesso zera o contador.        |
| Bloqueio temporário      | 30 s                   | Sem aceitar credenciais durante o período.            |
| Debounce do teclado      | 40 ms                  | Ajustável após medição no hardware.                   |
| Scan completo do teclado | 20 ms ou menos         | Uma linha por ciclo é preferível.                     |
| Timeout I2C              | 20 ms                  | Falha deve retornar ao chamador; sem loop infinito.   |

## 3. Visão do sistema e casos de uso

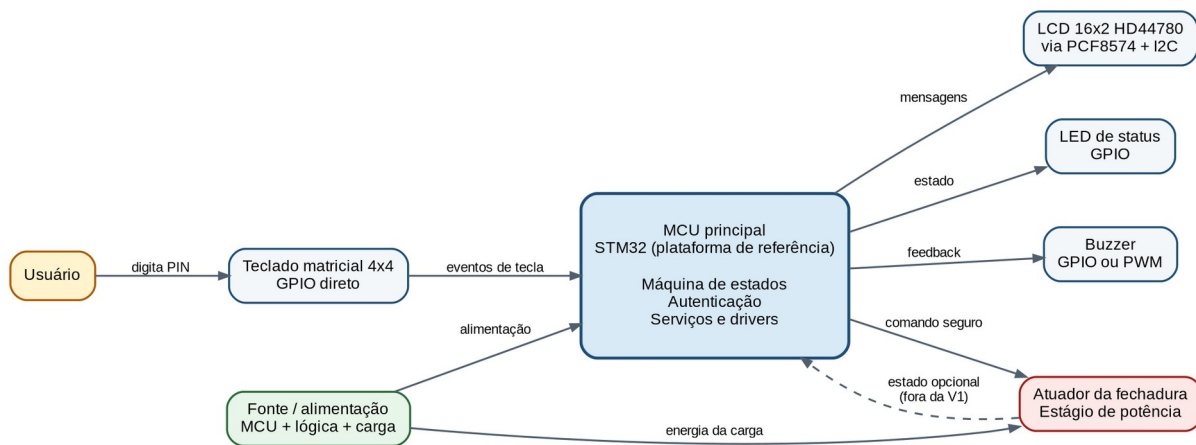


Figura 1 - Contexto físico e responsabilidades principais do sistema.

### 3.1 Atores

| Ator                  | Interação                                                 | Limites                                                          |
|-----------------------|-----------------------------------------------------------|------------------------------------------------------------------|
| Usuário               | Digita PIN, confirma, cancela e observa feedback.         | Não configura o sistema na V1.                                   |
| Técnico/desenvolvedor | Programa firmware, executa diagnóstico e valida hardware. | Interface de debug não é interface de usuário.                   |
| Fonte de alimentação  | Fornece energia à lógica e ao atuador.                    | Quedas e brownouts devem levar o sistema ao estado seguro.       |
| Mecanismo de trava    | Converte o comando elétrico em ação mecânica.             | Sem sensor de retorno na V1; estado é comandado, não confirmado. |

### 3.2 Caso de uso principal - acesso autorizado

18. Sistema inicia e mantém o atuador no estado seguro/travado.
19. LCD apresenta solicitação de PIN e LED indica estado travado.
20. Usuário digita quatro dígitos; o display mostra marcadores, não os números.
21. Usuário pressiona # para confirmar.
22. Serviço de autenticação compara a credencial candidata com o PIN configurado.
23. Se válida, o controlador aciona o atuador, sinaliza sucesso e inicia deadline de destravamento.
24. Ao expirar o tempo, o controlador desaciona o atuador e retorna ao estado travado.

### 3.3 Casos alternativos

| Caso                           | Comportamento esperado                                                                         |
|--------------------------------|------------------------------------------------------------------------------------------------|
| PIN inválido                   | Sinalizar erro, incrementar tentativas e permitir nova entrada ou entrar em bloqueio.          |
| Entrada incompleta + #         | Não autenticar; informar PIN incompleto e permanecer na entrada.                               |
| Tecla *                        | Limpar credencial em edição; se vazia, cancelar e voltar ao estado ocioso.                     |
| Timeout de digitação           | Apagar buffer, ocultar dados e retornar ao estado travado.                                     |
| Três falhas consecutivas       | Entrar em LOCKOUT por 30 s, ignorando dígitos e mantendo a trava segura.                       |
| Falha crítica de inicialização | Não acionar carga; entrar em FAULT e sinalizar condição de forma limitada.                     |
| Falha do LCD                   | A fechadura deve permanecer segura; decisão de permitir operação degradada deve ser explícita. |
| Reset durante destravamento    | Hardware e firmware devem levar o comando do atuador imediatamente ao estado seguro.           |

## 4. Requisitos funcionais

| ID     | Requisito                                                                                                                           | Justificativa                     | Critério de aceitação                                                                           |
|--------|-------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------|-------------------------------------------------------------------------------------------------|
| FR-001 | Ao iniciar, o sistema deve configurar a saída do atuador no estado seguro antes de habilitar qualquer lógica de aplicação.          | Evita pulso espúrio durante boot. | Em 20 ciclos de power-on/reset, o atuador não deve energizar inadvertidamente.                  |
| FR-002 | O sistema deve inicializar teclado, base de tempo, I2C, PCF8574, LCD, LED, buzzer e atuador, registrando o resultado de cada etapa. | Inicialização determinística.     | Falhas são detectadas e encaminhadas à política de fault; nenhum init bloqueia indefinidamente. |
| FR-003 | O teclado deve gerar um único evento de pressionamento por acionamento físico válido.                                               | Evita repetição por bouncing.     | Cada pressão normal produz exatamente um evento em 100 testes consecutivos.                     |
| FR-004 | Os dígitos 0 a 9 devem compor a credencial candidata até quatro posições.                                                           | Entrada do PIN.                   | Dígitos adicionais são ignorados ou sinalizados sem overflow.                                   |
| FR-005 | A tecla * deve limpar a entrada atual; com buffer vazio, deve cancelar a sessão de entrada.                                         | Recuperação simples do usuário.   | Buffer e display retornam ao estado definido sem resíduos.                                      |
| FR-006 | A tecla # deve solicitar autenticação somente quando a credencial possuir exatamente quatro dígitos.                                | Evita comparação ambígua.         | PIN incompleto nunca aciona o atuador.                                                          |

| ID     | Requisito                                                                                                          | Justificativa                          | Critério de aceitação                                                           |
|--------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------|---------------------------------------------------------------------------------|
| FR-007 | O LCD não deve exibir os dígitos reais do PIN.                                                                     | Reduz exposição visual.                | Somente marcadores como * são mostrados.                                        |
| FR-008 | O serviço de autenticação deve comparar a credencial candidata com o PIN fixo configurado.                         | Regra de acesso V1.                    | PIN correto retorna sucesso; qualquer divergência retorna falha.                |
| FR-009 | Uma autenticação válida deve zerar o contador de falhas consecutivas.                                              | Política previsível.                   | Após sucesso, a próxima falha é contabilizada como primeira.                    |
| FR-010 | Uma autenticação inválida deve incrementar o contador de falhas consecutivas.                                      | Base para lockout.                     | Contador satura no limite configurado e não sofre overflow.                     |
| FR-011 | Após três falhas consecutivas, o sistema deve entrar em bloqueio temporário por 30 s.                              | Mitiga tentativa automatizada simples. | Nenhum PIN é autenticado durante o lockout; saída permanece segura.             |
| FR-012 | Uma autenticação válida deve comandar o atuador para o estado destravado por 3 s.                                  | Função principal.                      | Pulso medido permanece dentro da tolerância definida e termina automaticamente. |
| FR-013 | O atuador deve retornar ao estado seguro ao expirar o deadline, independentemente do estado do LCD, LED ou buzzer. | Separação de segurança.                | Falhas de UI não prolongam acionamento.                                         |
| FR-014 | O LED deve indicar, no mínimo, travado, destravado, erro de autenticação, lockout e fault.                         | Observabilidade.                       | Cada estado possui padrão distinguível e documentado.                           |
| FR-015 | O buzzer deve emitir feedback curto para tecla válida, sucesso, erro e entrada em lockout.                         | Usabilidade.                           | Padrões não bloqueiam o superloop.                                              |
| FR-016 | A entrada de credencial deve expirar após 15 s sem tecla válida.                                                   | Evita sessão abandonada.               | Buffer é apagado e o estado retorna a LOCKED_IDLE.                              |
| FR-017 | O driver do LCD deve suportar inicialização, clear, cursor, escrita de caractere e string.                         | Conjunto mínimo de UI.                 | Todos os comandos são exercitados em teste de integração.                       |
| FR-018 | O adapter HD44780-PCF8574 deve preservar o estado do backlight ao transmitir nibbles e pulsos de enable.           | Integridade do mapeamento.             | Backlight não pisca durante escrita normal.                                     |
| FR-019 | Toda transação I2C deve possuir timeout finito e retornar status.                                                  | Evita deadlock do firmware.            | Desconectar o LCD não congela o sistema.                                        |
| FR-020 | Uma falha crítica deve levar a máquina de estados a FAULT, apagar credencial e manter o atuador seguro.            | Falha segura.                          | Injeção de erro crítico não produz destravamento.                               |

## 5. Requisitos não funcionais

| ID      | Requisito         | Justificativa                                                   | Critério de aceitação                                            |
|---------|-------------------|-----------------------------------------------------------------|------------------------------------------------------------------|
| NFR-001 | Determinismo      | Nenhuma operação de aplicação pode depender de espera infinita. | Análise e testes mostram deadlines finitos em I2C, atuação e UI. |
| NFR-002 | Tempo de resposta | Uma tecla válida deve produzir feedback perceptível em até 100  | Medido do contato estabilizado ao feedback.                      |

| ID      | Requisito                | Justificativa                                                                                             | Critério de aceitação                                          |
|---------|--------------------------|-----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
|         |                          | ms.                                                                                                       |                                                                |
| NFR-003 | Não bloqueio             | Delays maiores que o pulso elétrico estritamente necessário devem ser substituídos por estados/deadlines. | Busca estática e revisão não encontram HAL_Delay na aplicação. |
| NFR-004 | Segurança do atuador     | A saída deve possuir estado seguro em reset e limite máximo de energização.                               | Validado por reset, hang simulado e teste de power cycle.      |
| NFR-005 | Robustez a rollover      | Cálculos de tempo devem funcionar durante wrap-around do contador uint32_t.                               | Testes unitários próximos de UINT32_MAX passam.                |
| NFR-006 | Encapsulamento           | Estado interno de módulos não deve ser exposto como variável global pública.                              | Headers não exportam dados mutáveis desnecessários.            |
| NFR-007 | Testabilidade            | Máquina de estados, autenticação e política de tempo devem compilar em host sem STM32 HAL.                | Suite de testes roda no PC.                                    |
| NFR-008 | Portabilidade localizada | Troca do LCD backpack ou implementação do teclado não deve alterar lock_controller.                       | Dependências revisadas e mocks substituem drivers.             |
| NFR-009 | Observabilidade          | Falhas relevantes devem produzir status interno e indicação diagnóstica.                                  | Códigos de erro podem ser inspecionados no debug.              |
| NFR-010 | Uso de memória           | Buffers devem possuir tamanho estático conhecido; sem malloc/free na V1.                                  | Mapa de memória e revisão confirmam alocação estática.         |
| NFR-011 | Qualidade de API         | Funções públicas devem ter contrato, unidades, ownership e retorno documentados.                          | Revisão dos headers passa checklist.                           |
| NFR-012 | Build reproduzível       | Configuração de build e opções do MCU devem estar versionadas.                                            | Clone limpo compila sem passos ocultos.                        |
| NFR-013 | Separação de vendor code | Código gerado pelo CubeMX e código do projeto devem permanecer em áreas distintas.                        | Regeneração não sobrescreve módulos de aplicação.              |
| NFR-014 | Falha segura             | Na incerteza, o sistema deve preferir manter/trazer a trava ao estado seguro.                             | Cenários de erro não resultam em comando de abertura.          |
| NFR-015 | Documentação             | Requisitos, pinout, estados e interfaces devem acompanhar a versão do firmware.                           | Release contém documento e README atualizados.                 |
| NFR-016 | Simplicidade controlada  | Nenhuma camada ou interface genérica deve ser adicionada sem requisito, teste ou variação concreta.       | Revisão arquitetural justifica cada abstração.                 |

## 6. Máquina de estados e comportamento

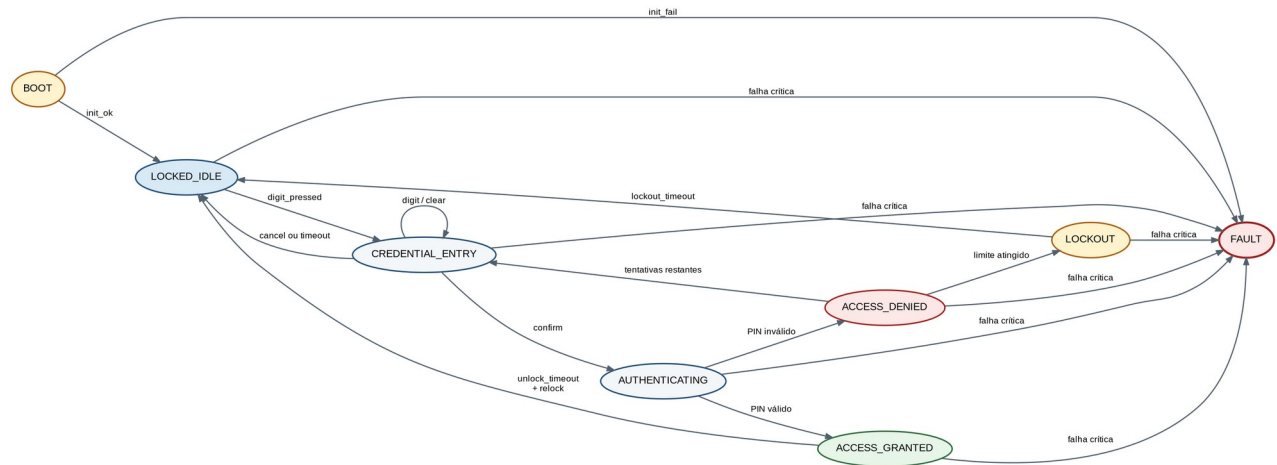


Figura 2 - Máquina de estados proposta para a V1.

### 6.1 Estados

| Estado           | Responsabilidade                                                        | Saída do atuador         | Entrada aceita             |
|------------------|-------------------------------------------------------------------------|--------------------------|----------------------------|
| BOOT             | Configurar hardware, validar inicialização e estabelecer estado seguro. | Seguro/desligado         | Nenhuma                    |
| LOCKED_IDLE      | Aguardar início de credencial; apresentar estado travado.               | Seguro/desligado         | Dígito inicia sessão       |
| CREDENTIAL_ENTRY | Capturar, mascarar, limpar e temporizar PIN candidato.                  | Seguro/desligado         | Dígitos, *, #              |
| AUTHENTICATING   | Executar comparação e atualizar política de tentativas.                 | Seguro/desligado         | Nenhuma                    |
| ACCESS_GRANTED   | Comandar abertura por deadline e informar sucesso.                      | Ativo por tempo limitado | Ignorada ou limitada       |
| ACCESS_DENIED    | Informar falha e decidir nova tentativa ou lockout.                     | Seguro/desligado         | Nenhuma durante indicação  |
| LOCKOUT          | Bloquear autenticação até expirar o prazo.                              | Seguro/desligado         | Somente eventos de sistema |
| FAULT            | Conter falha crítica, apagar dados e impedir abertura.                  | Seguro/desligado         | Reset/diagnóstico          |

### 6.2 Eventos

| Evento                    | Origem                          | Dados                                 |
|---------------------------|---------------------------------|---------------------------------------|
| EV_INIT_OK / EV_INIT_FAIL | Inicialização                   | Bitmap ou código de módulo que falhou |
| EV_KEY_DIGIT              | Credential input                | Dígito 0-9                            |
| EV_KEY_CONFIRM            | Credential input                | Nenhum                                |
| EV_KEY_CANCEL             | Credential input                | Nenhum                                |
| EV_ENTRY_TIMEOUT          | Time service                    | Nenhum                                |
| EV_AUTH_OK / EV_AUTH_FAIL | Authentication                  | Resultado                             |
| EV_UNLOCK_TIMEOUT         | Time service / actuator control | Nenhum                                |



| Evento             | Origem              | Dados           |
|--------------------|---------------------|-----------------|
| EV_LOCKOUT_TIMEOUT | Time service        | Nenhum          |
| EV_CRITICAL_FAULT  | Qualquer subsistema | Código de falha |

### 6.3 Invariantes obrigatórias

- Fora de ACCESS\_GRANTED, a saída lógica do atuador deve estar no estado seguro.
- Uma credencial candidata nunca pode exceder o buffer reservado.
- Ao sair de CREDENTIAL\_ENTRY, o buffer deve ser invalidado ou apagado.
- AUTHENTICATING não deve possuir side effects de hardware além do resultado retornado ao controlador.
- LOCKOUT não aceita autenticação, ainda que o teclado continue sendo escaneado.
- FAULT tem precedência sobre eventos de interface de usuário.
- O timeout do atuador não pode depender da conclusão de mensagens no LCD ou buzzer.

## 7. Arquitetura de hardware

### 7.1 Componentes e conexão

| Bloco            | Conexão proposta                    | Observações de engenharia                                                  |
|------------------|-------------------------------------|----------------------------------------------------------------------------|
| STM32F103C8T6    | Plataforma principal                | Referência de desenvolvimento; confirmar recursos e pinout final.          |
| Teclado 4x4      | 8 GPIOs: 4 linhas + 4 colunas       | Linhas dirigidas sequencialmente; colunas com pull-up adequado.            |
| PCF8574          | I2C + alimentação                   | Dedicado ao LCD na V1; endereço validado por A0-A2.                        |
| LCD HD44780 16x2 | 4 bits via PCF8574                  | RW preferencialmente em GND; mapear RS, E, D4-D7 e backlight.              |
| LED de status    | GPIO + resistor                     | Definir polaridade ativa no board_config.                                  |
| Buzzer ativo     | GPIO + estágio, se necessário       | Preferido na V1; buzzer passivo exige PWM e muda o driver.                 |
| Atuador          | GPIO -> driver de potência -> carga | MOSFET/transistor, flyback, corrente e dissipação devem ser dimensionados. |
| Fonte            | Domínios lógica e carga             | Desacoplamento, retorno de corrente e brownout são críticos.               |

### 7.2 Orçamento preliminar de pinos

| Função              | Quantidade  | Observação                                                                            |
|---------------------|-------------|---------------------------------------------------------------------------------------|
| Teclado matricial   | 8           | Pode consumir parte relevante dos GPIOs; intencional na V1 para reduzir complexidade. |
| I2C LCD             | 2           | SCL e SDA com pull-ups externos adequados.                                            |
| LED                 | 1           | Saída digital.                                                                        |
| Buzzer              | 1           | GPIO para ativo ou canal PWM para passivo.                                            |
| Atuador             | 1           | Saída lógica para estágio de potência.                                                |
| Debug SWD           | 2           | Reservar SWDIO/SWCLK.                                                                 |
| UART de diagnóstico | 2 opcionais | Útil no desenvolvimento; pode ser omitida do produto.                                 |

| Função     | Quantidade | Observação                                 |
|------------|------------|--------------------------------------------|
| Total base | 15         | Sem contar alimentação, reset e oscilador. |

### Decisão para a V1

Não compartilhar o PCF8574 do LCD com o teclado. O teclado permanece em GPIO direto. Essa escolha reduz risco de read-modify-write, simplifica o scan e isola falhas do display da entrada de credencial.

## 7.3 Requisitos elétricos do atuador

- O GPIO do MCU nunca deve alimentar diretamente solenoide, relé ou motor.
- O estágio deve ser dimensionado para corrente de partida, corrente contínua e dissipação térmica.
- Carga indutiva exige caminho de recirculação, como diodo flyback, compatível com o tempo de desenergização desejado.
- A saída deve ter pull-down ou pull-up de hardware que preserve o estado seguro durante reset e boot.
- A alimentação da carga deve possuir retorno planejado para não injetar ruído no domínio lógico.
- O limite de 3 s é parâmetro de software, não substitui proteção elétrica/térmica apropriada.

## 7.4 Decisões elétricas ainda obrigatórias

| Decisão                    | Por que bloqueia o projeto                                   | Saída esperada                                                      |
|----------------------------|--------------------------------------------------------------|---------------------------------------------------------------------|
| Tipo exato do atuador      | Define circuito, corrente, polaridade e interface do driver. | Part number, tensão, corrente, duty cycle e mecanismo de fail-safe. |
| Tipo de buzzer             | Define GPIO simples ou PWM.                                  | Ativo/passivo, tensão e corrente.                                   |
| Mapeamento do backpack LCD | Varia entre módulos comerciais.                              | Tabela PCF8574 bit -> RS/RW/E/BL/D4-D7.                             |
| Tensão do sistema          | Impacta níveis lógicos, LCD, PCF8574 e carga.                | Diagrama de alimentação e compatibilidade de níveis.                |
| Pull-ups I2C               | Afetam rise time e corrente.                                 | Valores calculados/medidos para o barramento real.                  |

## 8. Arquitetura de software e regras de dependência

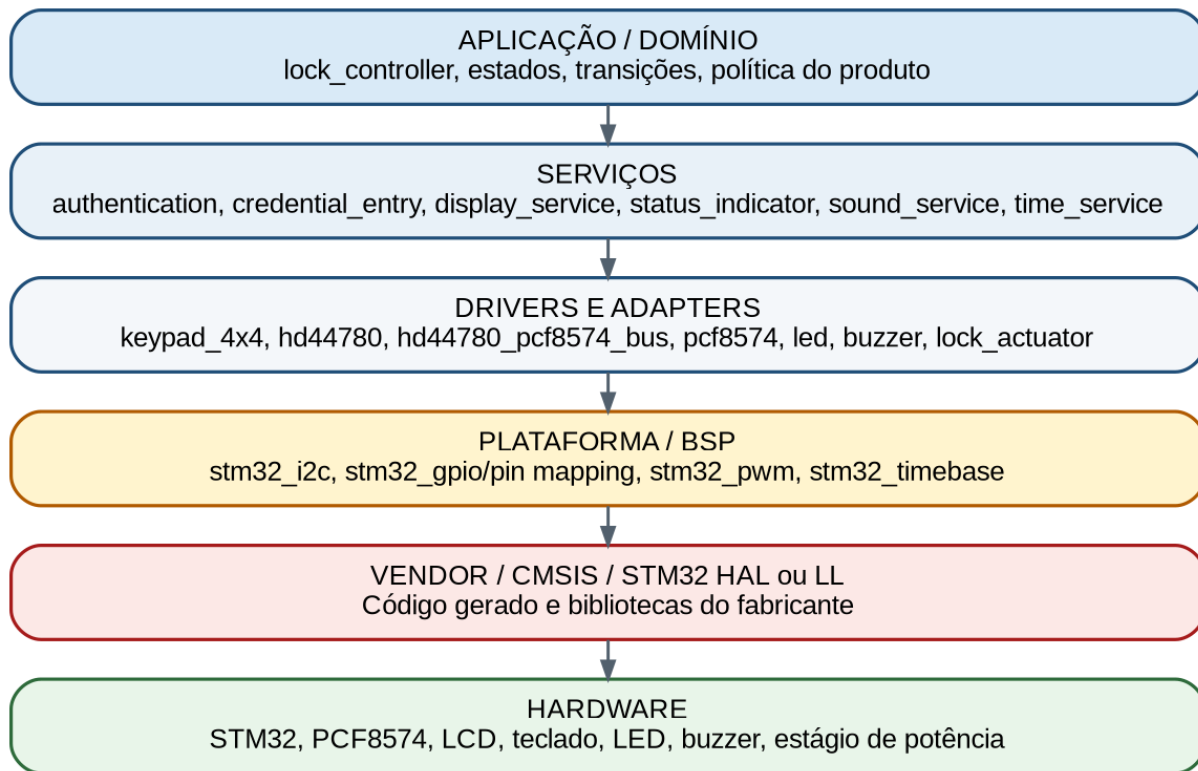


Figura 3 - Camadas e direção permitida das dependências.

### 8.1 Regra de dependência

Dependências apontam de cima para baixo. Módulos de alto nível não incluem headers da STM32 HAL nem conhecem endereços I2C, GPIO ports ou registradores. Um módulo inferior pode implementar uma interface exigida por uma camada superior, mas a política do produto permanece no domínio.

| Camada            | Pode conhecer                                | Não deve conhecer                              |
|-------------------|----------------------------------------------|------------------------------------------------|
| Aplicação/domínio | Serviços e tipos de domínio                  | HAL, PCF8574, HD44780, GPIO, I2C               |
| Serviços          | Drivers abstratos e tipos de serviço         | Fluxos completos da máquina de estados         |
| Drivers/adapters  | Interfaces de plataforma e componente físico | Regras como número de tentativas ou PIN válido |
| Plataforma/BSP    | HAL/LL, pinos, timers e periféricos STM32    | Semântica de acesso do produto                 |
| Vendor            | Hardware e bibliotecas do fabricante         | Política do projeto                            |

### 8.2 Cadeia do LCD



Figura 4 - Separação entre semântica do LCD, adaptação ao backpack e transporte I2C.

- **display\_service**: decide qual mensagem e formato a interface de usuário requer.
- **hd44780**: conhece comandos, endereçamento DDRAM, inicialização e temporização do controlador LCD.
- **hd44780\_pcf8574\_bus**: conhece o mapeamento elétrico e gera pulsos de E preservando backlight.
- **pcf8574**: expõe leitura/escrita de porta de 8 bits e não conhece LCD.
- **stm32\_i2c**: realiza transações com timeout e converte status da HAL para status do projeto.

**Correção de nomenclatura**

O PCF8574 não é um tradutor de comandos do LCD. Ele é um expensor de I/O. O adapter HD44780-PCF8574 é quem traduz nibbles e sinais do LCD em bytes enviados ao expensor.

**8.3 Critério para criar interfaces**

- Existe mais de uma implementação plausível durante a vida do produto.
- A interface permite testar lógica sem hardware.
- A dependência específica teria alto custo de propagação.
- O módulo representa uma capacidade estável do sistema.
- O contrato pode ser descrito sem vaziar detalhes do componente concreto.

Não criar wrappers que apenas renomeiam uma única função da HAL sem acrescentar contrato, política, conversão de erro ou isolamento de plataforma.

**9. Estrutura de pastas**

```

project/
├── App/
│   ├── Inc/
│   │   ├── app_main.h
│   │   ├── lock_controller.h
│   │   └── app_config.h
│   └── Src/
│       ├── app_main.c
│       └── lock_controller.c
├── Services/
│   ├── Inc/
│   │   ├── authentication.h
│   │   ├── credential_entry.h
│   │   ├── display_service.h
│   │   ├── status_indicator.h
│   │   ├── sound_service.h
│   │   └── time_service.h
│   └── Src/
├── Drivers/
│   ├── Keypad4x4/
│   ├── HD44780/
│   ├── PCF8574/
│   ├── Led/
│   ├── Buzzer/
│   └── LockActuator/
├── Platform/
│   ├── Inc/
│   │   ├── board_config.h
│   │   ├── stm32_i2c_port.h
│   │   ├── stm32_timebase.h
│   │   └── stm32_pwm_port.h
│   └── Src/
├── Core/                                # CubeMX / startup / main gerado
├── Middlewares/                         # se necessário; vazio na V1
├── Tests/
│   ├── Host/
│   │   ├── test_lock_controller.c
│   │   ├── test_authentication.c
│   │   └── test_time_service.c
│   └── Target/
├── Docs/
│   ├── architecture/
│   ├── hardware/
│   └── test_reports/
├── CMakeLists.txt
└── README.md

```

**Regra para CubeMX**

Core e arquivos gerados são tratados como plataforma. A aplicação real não deve ser implementada dentro de regiões USER CODE do main.c além da inicialização mínima e da chamada para App\_Init/App\_Run. Regenerar o .ioc não pode destruir o software do produto.

**9.1 Convenções de organização**

- Cada módulo possui um header público mínimo e um source com dados privados static.
- Headers privados adicionais ficam dentro da pasta do módulo, não no include global.
- app\_config.h contém políticas configuráveis; board\_config.h contém pinos, periféricos e polaridades.
- Não duplicar tipos entre camadas; tipos de domínio pertencem à camada que define seu significado.
- Código de teste não inclui main de produção nem depende de hardware quando o teste é de lógica.
- Documentos e resultados de teste são versionados junto do código quando representam a baseline.

**10. Responsabilidades dos módulos****10.1 Aplicação e domínio****app\_main**

|                  |                                                    |
|------------------|----------------------------------------------------|
| <b>Camada</b>    | Aplicação                                          |
| <b>Propósito</b> | Ponto de composição do software e ciclo principal. |

**Responsabilidades:**

- Inicializar módulos na ordem definida.
- Construir dependências/handles.
- Executar o superloop e despachar atualizações.
- Encaminhar falhas de inicialização ao lock\_controller/fault policy.

**Não deve fazer:**

- Não implementar autenticação.
- Não manipular GPIO/I2C diretamente.
- Não conter delays de comportamento.

**Dependências permitidas:** Interfaces públicas dos serviços e drivers necessários à composição.

**Interface pública sugerida:**

```
AppStatus App_Init(void);
void App_Run(void);
```

**Testes mínimos:** Teste de smoke no alvo; verificação de ordem de inicialização e comportamento com falha injetada.

**lock\_controller**

|                  |                                                     |
|------------------|-----------------------------------------------------|
| <b>Camada</b>    | Aplicação/domínio                                   |
| <b>Propósito</b> | Máquina de estados e política central da fechadura. |

**Responsabilidades:**

- Manter o estado do sistema.
- Consumir eventos de credencial e tempo.
- Solicitar autenticação.
- Comandar serviços de UI e atuador.
- Aplicar política de tentativas e lockout.
- Priorizar falhas críticas.

**Não deve fazer:**

- Não escanear teclado.
- Não formatar bytes do LCD.

- Não conhecer HAL ou pinos.
- Não armazenar a senha válida.

**Dependências permitidas:** authentication, credential\_entry, display\_service, status\_indicator, sound\_service, lock\_actuator e time\_service.

**Interface pública sugerida:**

```
LockStatus LockController_Init(LockController *self, const LockDeps *deps);
void LockController_Update(LockController *self, uint32_t now_ms);
LockState LockController_GetState(const LockController *self);
```

**Testes mínimos:** Cobertura de todas as transições, eventos simultâneos, timeouts, lockout, fault e rollover.

### app\_config

|                  |                                                        |
|------------------|--------------------------------------------------------|
| <b>Camada</b>    | Configuração de aplicação                              |
| <b>Propósito</b> | Centralizar políticas da V1 em constantes versionadas. |

**Responsabilidades:**

- Definir duração de unlock, lockout, timeout de entrada, tamanho do PIN e padrões de indicação.
- Validar limites por static\_assert quando aplicável.

**Não deve fazer:**

- Não conter pinout físico.
- Não expor credencial em logs.
- Não misturar parâmetros de HAL.

**Dependências permitidas:** Nenhuma; deve ser consumido pelas camadas de política.

**Interface pública sugerida:**

```
#define APP_PIN_LENGTH          (4U)
#define APP_ENTRY_TIMEOUT_MS    (15000U)
#define APP_UNLOCK_DURATION_MS  (3000U)
#define APP_LOCKOUT_DURATION_MS (30000U)
#define APP_MAX_FAILED_ATTEMPTS (3U)
```

**Testes mínimos:** Build-time checks e revisão de consistência dos valores.

## 10.2 Serviços

### authentication

|                  |                                                                  |
|------------------|------------------------------------------------------------------|
| <b>Camada</b>    | Serviço de domínio                                               |
| <b>Propósito</b> | Validar uma credencial candidata contra a credencial fixa da V1. |

**Responsabilidades:**

- Receber buffer e comprimento explícitos.
- Comparar exatamente quatro dígitos.
- Retornar resultado sem side effects de hardware.
- Evitar leitura fora dos limites.

**Não deve fazer:**

- Não contar tentativas.
- Não controlar lockout.
- Não comandar atuador ou display.
- Não acessar teclado.

**Dependências permitidas:** Somente configuração segura da credencial.

**Interface pública sugerida:**

```
AuthResult Authentication_Verify(const uint8_t *candidate, size_t length);
```

**Testes mínimos:** PIN correto, cada posição incorreta, tamanhos 0-8, ponteiro inválido conforme contrato.

### credential\_entry

|                  |                                                                                        |
|------------------|----------------------------------------------------------------------------------------|
| <b>Camada</b>    | Serviço                                                                                |
| <b>Propósito</b> | Converter eventos físicos do teclado em uma credencial candidata e eventos semânticos. |

#### Responsabilidades:

- Manter buffer limitado.
- Interpretar dígitos, # e \*.
- Gerenciar timeout de entrada.
- Mascara a representação para UI.
- Apagar buffer ao finalizar/cancelar.

#### Não deve fazer:

- Não decidir se a credencial é válida.
- Não controlar tentativas.
- Não escrever diretamente no LCD.

**Dependências permitidas:** keypad\_4x4/event source e time\_service.

#### Interface pública sugerida:

```
void CredentialEntry_Reset(CredentialEntry *self);
CredentialEvent CredentialEntry_ProcessKey(CredentialEntry *self, KeyCode key, uint32_t now_ms);
bool CredentialEntry_HasTimedOut(const CredentialEntry *self, uint32_t now_ms);
```

**Testes mínimos:** Limites do buffer, backspace/clear, confirm incompleto, timeout e rollover.

### display\_service

|                  |                                                                |
|------------------|----------------------------------------------------------------|
| <b>Camada</b>    | Serviço de UI                                                  |
| <b>Propósito</b> | Apresentar mensagens do produto sem expor detalhes do HD44780. |

#### Responsabilidades:

- Traduzir estados em telas curtas.
- Evitar reescritas desnecessárias.
- Mascara o PIN.
- Permitir modo degradado se política permitir.

#### Não deve fazer:

- Não realizar I2C.
- Não decidir transições.
- Não armazenar a credencial real.

**Dependências permitidas:** hd44780 driver.

#### Interface pública sugerida:

```
DisplayStatus Display_Init(void);
void Display_ShowLocked(void);
void Display_ShowPinEntry(uint8_t digits_entered);
void Display_ShowGranted(void);
void Display_ShowDenied(uint8_t remaining);
void Display_ShowLockout(uint32_t remaining_s);
void Display_ShowFault(FaultCode code);
```

**Testes mínimos:** Teste de formatação com fake LCD e teste visual no alvo.

### status\_indicator

|                  |                                                                |
|------------------|----------------------------------------------------------------|
| <b>Camada</b>    | Serviço de UI                                                  |
| <b>Propósito</b> | Mapear estados do produto para padrões de LED não bloqueantes. |

**Responsabilidades:**

- Definir padrões temporais.
- Atualizar saída com base em now\_ms.
- Permitir prioridade de fault sobre padrões normais.

**Não deve fazer:**

- Não manipular GPIO diretamente.
- Não conter lógica de autenticação.

**Dependências permitidas:** led driver e time\_service.

**Interface pública sugerida:**

```
void StatusIndicator_SetMode(StatusMode mode, uint32_t now_ms);
void StatusIndicator_Update(uint32_t now_ms);
```

**Testes mínimos:** Sequência temporal, troca de modo, prioridade e rollover.

**sound\_service**

|                  |                                                  |
|------------------|--------------------------------------------------|
| <b>Camada</b>    | Serviço de UI                                    |
| <b>Propósito</b> | Mapear eventos do produto em sequências sonoras. |

**Responsabilidades:**

- Executar beep curto, sucesso, erro e lockout sem bloquear.
- Interromper ou priorizar padrões segundo política.

**Não deve fazer:**

- Não decidir autenticação.
- Não acessar timer/PWM diretamente.

**Dependências permitidas:** buzzer driver e time\_service.

**Interface pública sugerida:**

```
void Sound_Play(SoundPattern pattern, uint32_t now_ms);
void Sound_Update(uint32_t now_ms);
void Sound_Stop(void);
```

**Testes mínimos:** Duração, sequência, interrupção e ausência de bloqueio.

**time\_service**

|                  |                                                      |
|------------------|------------------------------------------------------|
| <b>Camada</b>    | Infraestrutura                                       |
| <b>Propósito</b> | Fornecer tempo monotônico e operações rollover-safe. |

**Responsabilidades:**

- Expor now\_ms.
- Verificar elapsed/deadline com aritmética unsigned.
- Padronizar unidades.

**Não deve fazer:**

- Não possuir um timer de software por feature na primeira versão.
- Não chamar HAL\_Delay na aplicação.

**Dependências permitidas:** stm32\_timebase.

**Interface pública sugerida:**

```
uint32_t Time_NowMs(void);
bool Time_Elapsed(uint32_t now, uint32_t start, uint32_t duration_ms);
```

**Testes mínimos:** Rollover, duração zero, limites e monotonicidade.



## 10.3 Drivers e adapters

### keypad\_4x4

|           |                                                               |
|-----------|---------------------------------------------------------------|
| Camada    | Driver de dispositivo                                         |
| Propósito | Escanear a matriz elétrica e gerar eventos de tecla estáveis. |

#### Responsabilidades:

- Dirigir uma linha por vez.
- Ler colunas.
- Aplicar debounce.
- Mapear linha/coluna para KeyCode.
- Detectar press e release.

#### Não deve fazer:

- Não montar PIN.
- Não interpretar # como autenticação.
- Não conhecer lockout.

**Dependências permitidas:** board GPIO access e time\_service.

#### Interface pública sugerida:

```
KeypadStatus Keypad_Init(Keypad *self);
void Keypad_Update(Keypad *self, uint32_t now_ms);
bool Keypad_GetEvent(Keypad *self, KeyEvent *event);
```

**Testes mínimos:** Cada tecla, múltiplas teclas, bouncing, tecla mantida, ghosting documentado.

### hd44780

|           |                                                                        |
|-----------|------------------------------------------------------------------------|
| Camada    | Driver de dispositivo                                                  |
| Propósito | Implementar protocolo e comandos do controlador LCD em modo de 4 bits. |

#### Responsabilidades:

- Executar sequência de inicialização.
- Enviar command/data.
- Controlar cursor e DDRAM.
- Escrever strings limitadas.
- Respeitar temporizações.

#### Não deve fazer:

- Não conhecer I2C.
- Não conhecer PCF8574.
- Não decidir textos do produto.

**Dependências permitidas:** Interface de barramento LCD de 4 bits e timebase de curta duração.

#### Interface pública sugerida:

```
LcdStatus Hd44780_Init(Hd44780 *self);
LcdStatus Hd44780_Clear(Hd44780 *self);
LcdStatus Hd44780_SetCursor(Hd44780 *self, uint8_t row, uint8_t col);
LcdStatus Hd44780_Write(Hd44780 *self, const char *text);
```

**Testes mínimos:** Inicialização, limites 16x2, comandos, falhas propagadas e strings longas.

### hd44780\_pcf8574\_bus

|           |                                                                          |
|-----------|--------------------------------------------------------------------------|
| Camada    | Adapter                                                                  |
| Propósito | Converter operações de nibble do HD44780 em estados de porta do PCF8574. |

**Responsabilidades:**

- Mapear RS/E/D4-D7/BL.
- Gerar pulso de enable.
- Preservar backlight.
- Forçar RW conforme hardware.
- Propagar erro de escrita.

**Não deve fazer:**

- Não conhecer comandos Clear/Home.
- Não formatar texto.
- Não realizar HAL I2C diretamente.

**Dependências permitidas:** pcf8574 driver e delay curto controlado.

**Interface pública sugerida:**

```
LcdBusStatus LcdBus_WriteNibble(LcdBus *bus, uint8_t nibble, bool rs);
LcdBusStatus LcdBus_SetBacklight(LcdBus *bus, bool on);
```

**Testes mínimos:** Todos os mapas de bits, sequência E low-high-low, preservação de BL e falha I2C.

**pcf8574**

|                  |                                                             |
|------------------|-------------------------------------------------------------|
| <b>Camada</b>    | Driver de dispositivo                                       |
| <b>Propósito</b> | Controlar a porta quasi-bidirecional de 8 bits do expensor. |

**Responsabilidades:**

- Validar endereço.
- Escrever byte.
- Ler byte quando necessário.
- Manter shadow apenas se o contrato exigir.

**Não deve fazer:**

- Não conhecer LCD.
- Não gerar pulsos de enable.
- Não conter política de retry infinita.

**Dependências permitidas:** stm32\_i2c\_port.

**Interface pública sugerida:**

```
PcfStatus Pcf8574_Init(Pcf8574 *self, uint8_t address_7bit);
PcfStatus Pcf8574_Write(Pcf8574 *self, uint8_t value);
PcfStatus Pcf8574_Read(Pcf8574 *self, uint8_t *value);
```

**Testes mínimos:** Endereço, NACK, timeout, desconexão e bytes 0x00/0xFF.

**led**

|                  |                                                         |
|------------------|---------------------------------------------------------|
| <b>Camada</b>    | Driver simples                                          |
| <b>Propósito</b> | Controlar o nível físico do LED respeitando polaridade. |

**Responsabilidades:**

- Inicializar estado seguro.
- Set on/off.
- Opcionalmente toggle.

**Não deve fazer:**

- Não implementar padrões temporais.
- Não conhecer estados da fechadura.

**Dependências permitidas:** board\_config/GPIO.

**Interface pública sugerida:**

```
void Led_Init(Led *self);
void Led_Set(Led *self, bool on);
```

**Testes mínimos:** Polaridade ativa alta/baixa e estado após init.

### buzzer

|                  |                                       |
|------------------|---------------------------------------|
| <b>Camada</b>    | Driver simples                        |
| <b>Propósito</b> | Controlar o transdutor sonoro físico. |

#### Responsabilidades:

- Para ativo: on/off.
- Para passivo: start/stop PWM com frequência definida.
- Iniciar desligado.

#### Não deve fazer:

- Não gerar melodias/padrões.
- Não bloquear por duração.

**Dependências permitidas:** GPIO ou stm32\_pwm\_port.

#### Interface pública sugerida:

```
BuzzerStatus Buzzer_Start(Buzzer *self, uint16_t frequency_hz);
void Buzzer_Stop(Buzzer *self);
```

**Testes mínimos:** On/off, frequência válida se passivo e estado seguro.

### lock\_actuator

|                  |                                                        |
|------------------|--------------------------------------------------------|
| <b>Camada</b>    | Driver com requisito de segurança                      |
| <b>Propósito</b> | Controlar o estágio de potência e limitar energização. |

#### Responsabilidades:

- Inicializar desativado.
- Comandar lock/unlock.
- Manter deadline máximo interno ou redundante.
- Forçar safe-off em fault.
- Expor estado comandado.

#### Não deve fazer:

- Não autenticar.
- Não depender do LCD.
- Não permitir acionamento sem limite temporal na API da V1.

**Dependências permitidas:** GPIO de board e time\_service.

#### Interface pública sugerida:

```
ActuatorStatus LockActuator_Init(LockActuator *self);
ActuatorStatus LockActuator_UnlockFor(LockActuator *self, uint32_t now_ms, uint32_t duration_ms);
void LockActuator_Update(LockActuator *self, uint32_t now_ms);
void LockActuator_ForceSafe(LockActuator *self);
```

**Testes mínimos:** Init, timeout, reset lógico, duração máxima, chamadas repetidas e fault.

## 10.4 Plataforma/BSP

### stm32\_i2c\_port

|                  |                                                                    |
|------------------|--------------------------------------------------------------------|
| <b>Camada</b>    | Plataforma                                                         |
| <b>Propósito</b> | Adaptar a API de I2C da STM32 para um contrato estável do projeto. |

**Responsabilidades:**

- Transmitir/receber com endereço 7-bit documentado.
- Aplicar timeout finito.
- Converter HAL\_StatusTypeDef em status do projeto.
- Centralizar handle do periférico.

**Não deve fazer:**

- Não conhecer PCF8574.
- Não escanear automaticamente o barramento em produção.
- Não fazer retry infinito.

**Dependências permitidas:** STM32 HAL/LL.

**Interface pública sugerida:**

```
IoStatus I2cPort_Write(uint8_t address_7bit, const uint8_t *data, size_t length, uint32_t timeout_ms);
IoStatus I2cPort_Read(uint8_t address_7bit, uint8_t *data, size_t length, uint32_t timeout_ms);
```

**Testes mínimos:** ACK/NACK, timeout, parâmetros inválidos e recuperação após erro.

**stm32\_timebase**

|                  |                                                             |
|------------------|-------------------------------------------------------------|
| <b>Camada</b>    | Plataforma                                                  |
| <b>Propósito</b> | Obter tick monotônico de 1 ms a partir de SysTick ou timer. |

**Responsabilidades:**

- Inicializar fonte.
- Retornar contador uint32\_t.
- Documentar atomicidade.

**Não deve fazer:**

- Não implementar políticas de timeout do produto.
- Não redefinir o tick da HAL sem decisão explícita.

**Dependências permitidas:** CMSIS/HAL ou registrador de timer.

**Interface pública sugerida:**

```
uint32_t Stm32Timebase_GetMs(void);
```

**Testes mínimos:** Frequência, monotonicidade, interrupção e rollover.

**board\_config**

|                  |                                                                   |
|------------------|-------------------------------------------------------------------|
| <b>Camada</b>    | Plataforma/configuração                                           |
| <b>Propósito</b> | Concentrar pinout, polaridades, handles e mapeamento do hardware. |

**Responsabilidades:**

- Definir GPIO port/pin.
- Mapear bits do backpack.
- Definir endereço I2C.
- Registrar revisão de hardware.

**Não deve fazer:**

- Não conter política de PIN, lockout ou mensagens.
- Não ser incluído pela aplicação.

**Dependências permitidas:** Headers gerados do STM32 e definições da placa.

**Interface pública sugerida:**

```
#define BOARD_PCF8574_ADDRESS_7BIT    (0x27U)
#define BOARD_LED_ACTIVE_LEVEL        (GPIO_PIN_SET)
/* Demais pinos e mapeamentos da revisão */
```

**Testes mínimos:** Compilação por revisão e revisão cruzada com esquemático.

## 11. Modelo de execução, tempo e concorrência



Figura 5 - Ciclo cooperativo sugerido para a V1.

### 11.1 Superloop cooperativo

A V1 não requer RTOS. Cada módulo recebe atualizações curtas e retorna rapidamente. O estado fica dentro de estruturas de contexto. O superloop não espera por duração humana; ele compara timestamps e deixa os demais módulos progredirem.

```

App_Run():
    now = Time_NowMs()
    Keypad_Update(now)
    coletar eventos de tecla
    LockController_Update(now)
    StatusIndicator_Update(now)
    Sound_Update(now)
    LockActuator_Update(now)
    diagnóstico opcional
  
```

### 11.2 Política de bloqueio

| Operação             | Permitido bloquear? | Regra                                                                      |
|----------------------|---------------------|----------------------------------------------------------------------------|
| Pulso E do LCD       | Sim, muito curto    | Apenas a duração elétrica necessária, preferencialmente em microssegundos. |
| Inicialização do LCD | Limitado            | Pode conter sequência curta controlada durante boot; nunca loop infinito.  |
| Transação I2C        | Somente até timeout | A chamada síncrona deve retornar em tempo finito.                          |
| Debounce             | Não                 | Modelado por timestamp/estado.                                             |
| LED/buzzer           | Não                 | Padrões por state machine local.                                           |
| Tempo destravado     | Não                 | Deadline monitorado a cada ciclo.                                          |
| Lockout              | Não                 | Estado com deadline, mantendo o loop ativo.                                |

### 11.3 Aritmética de tempo rollover-safe

Use diferença unsigned, não comparação absoluta de timestamps futuros:

```

bool Time_Elapsed(uint32_t now, uint32_t start, uint32_t duration)
{
    return (uint32_t)(now - start) >= duration;
}
  
```

#### Restrição

A duração comparada deve ser menor que metade do espaço do contador quando se usa certas formas de comparação. Para os tempos da V1, essa condição é amplamente satisfeita. Documente a unidade em todos os parâmetros.

## 11.4 Interrupções

- A ISR da base de tempo deve apenas atualizar o tick necessário.
- Não escrever no LCD, não autenticar e não executar scan completo de teclado dentro de ISR.
- Eventos vindos de ISR devem ser flags/contadores simples, com acesso atômico documentado.
- Na V1, o teclado pode ser totalmente polling; isso reduz concorrência e facilita teste.
- Se I2C interrupt/DMA for adotado futuramente, a API e ownership do buffer precisam ser redesenhados conscientemente.

## 12. Estratégia de erros, falhas e segurança

### 12.1 Classificação de erros

| Classe                          | Exemplos                                                             | Resposta                                                   |
|---------------------------------|----------------------------------------------------------------------|------------------------------------------------------------|
| Erro de entrada                 | PIN incompleto, tecla inválida                                       | Informar usuário; não entrar em FAULT.                     |
| Erro recuperável de dispositivo | NACK ocasional, LCD indisponível                                     | Retornar status, limitar retries, considerar UI degradada. |
| Falha crítica                   | Atuador não inicializa, estado interno inválido, corrupção detectada | Forçar safe state e entrar em FAULT.                       |
| Falha de segurança              | Deadline do atuador violado, comando incoerente                      | Desativar saída imediatamente e registrar fault.           |

### 12.2 Modelo de status

- Cada driver retorna enum próprio ou status comum pequeno e documentado.
- Não usar bool quando é necessário distinguir timeout, NACK, parâmetro inválido e hardware não inicializado.
- Conversão de HAL\_StatusTypeDef deve ocorrer na camada de plataforma.
- A aplicação recebe erros com semântica útil; não deve interpretar códigos da HAL.
- Falhas persistentes devem ser latched quando necessário para diagnóstico.

```
typedef enum
{
    IO_STATUS_OK = 0,
    IO_STATUS_INVALID_ARGUMENT,
    IO_STATUS_NOT_INITIALIZED,
    IO_STATUS_TIMEOUT,
    IO_STATUS_NACK,
    IO_STATUS_HW_ERROR
} IoStatus;
```

### 12.3 Política de segurança da credencial

- O PIN não é exibido no LCD nem transmitido em logs.
- O buffer candidato é apagado ao confirmar, cancelar, expirar, obter sucesso, falhar ou entrar em fault.
- A comparação deve considerar comprimento e todos os dígitos esperados.
- O contador de tentativas é saturado no limite; não deve sofrer wrap-around.
- Durante LOCKOUT, eventos de confirmação são ignorados e não reiniciam o prazo.
- O PIN fixo em firmware é uma limitação explícita; não é proteção adequada para produto comercial.

### 12.4 Fail-safe do atuador

- Estado elétrico seguro antes da configuração do GPIO.
- Saída explicitamente desativada no início de LockActuator\_Init.
- API de abertura exige duração; evitar função pública Unlock() sem limite.
- Deadline local no driver atua como segunda barreira, além da máquina de estados.
- Fault e reset lógico chamam ForceSafe.
- Watchdog de hardware é recomendável após a lógica básica estar estável, mas não substitui o timeout do driver.

**Ponto cego relevante**

Sem sensor de posição, o firmware conhece apenas o comando enviado, não o estado mecânico real. A V1 não pode afirmar que a porta está efetivamente travada ou destravada; apenas que comandou o atuador.

## 13. Estratégia de testes e validação

### 13.1 Pirâmide de testes

| Nível      | Objetivo                   | Ambiente                    | Exemplos                                     |
|------------|----------------------------|-----------------------------|----------------------------------------------|
| Unitário   | Validar lógica isolada     | Host PC                     | auth, buffer, transições, timeouts, rollover |
| Componente | Validar driver isolado     | STM32 + periférico          | keypad, LCD, PCF8574, buzzer, LED            |
| Integração | Validar cadeias completas  | Hardware alvo               | display_service -> LCD; PIN -> unlock        |
| Sistema    | Validar requisitos         | Montagem completa           | casos de uso e falhas                        |
| Regressão  | Evitar retorno de defeitos | CI + alvo quando disponível | suíte automática e checklist                 |

### 13.2 Testes unitários mínimos

- Authentication: PIN correto, todos os dígitos incorretos individualmente, tamanhos inválidos.
- CredentialEntry: limite de 4 dígitos, clear, cancel, confirm incompleto, timeout e limpeza.
- LockController: todas as transições válidas e eventos ignorados por estado.
- Lockout: terceira falha entra em bloqueio; prazo expira; sucesso zera contador.
- Actuator policy: deadline sempre encerra abertura; fault força estado seguro.
- TimeService: wrap-around próximo de UIN32\_MAX.
- StatusIndicator e SoundService: sequências temporais e troca de prioridade.

### 13.3 Testes de drivers no alvo

| Driver      | Teste                                                                       | Instrumentação                                                   |
|-------------|-----------------------------------------------------------------------------|------------------------------------------------------------------|
| Keypad      | Pressionar cada tecla 100 vezes; manter tecla; pressionar duas simultâneas. | UART de debug ou contador observado; analisador lógico opcional. |
| I2C/PCF8574 | ACK, NACK, desconexão, endereço errado, recuperação.                        | Analisador lógico I2C.                                           |
| LCD         | Init a frio/quente, clear, duas linhas, texto limite, backlight.            | Inspeção visual + I2C capture.                                   |
| LED/Buzzer  | Polaridade, duração e padrões.                                              | Osciloscópio/analisador lógico.                                  |
| Atuador     | Pulso, reset durante pulso, limite térmico e ruído.                         | Osciloscópio, fonte limitada e medição de corrente.              |

### 13.4 Matriz de aceitação da V1

| ID    | Cenário         | Procedimento        | Resultado esperado                                     |
|-------|-----------------|---------------------|--------------------------------------------------------|
| AT-01 | Power-on seguro | Energizar 20 vezes. | Nenhum pulso de abertura; estado LOCKED_IDLE ou FAULT. |
| AT-02 | PIN correto     | Digitar PIN + #.    | Feedback de sucesso, abertura                          |

| ID    | Cenário                | Procedimento                            | Resultado esperado                                                              |
|-------|------------------------|-----------------------------------------|---------------------------------------------------------------------------------|
|       |                        |                                         | por 3 s e retorno seguro.                                                       |
| AT-03 | PIN errado             | Digitar quatro dígitos errados.         | Sem abertura; contador incrementado; feedback de erro.                          |
| AT-04 | Lockout                | Executar três falhas.                   | Bloqueio por 30 s; nenhum PIN aceito.                                           |
| AT-05 | Timeout de entrada     | Digitar parte do PIN e aguardar 15 s.   | Buffer apagado e retorno a LOCKED_IDLE.                                         |
| AT-06 | LCD desconectado       | Desconectar backpack antes do boot.     | Sem travamento de firmware; atuador seguro; fault/degradação conforme política. |
| AT-07 | Reset durante abertura | Resetar MCU com atuador ativo.          | Saída retorna imediatamente ao estado seguro.                                   |
| AT-08 | Bouncing               | Aplicar pressão física irregular.       | Um evento por pressionamento normal.                                            |
| AT-09 | Rollover               | Simular tick próximo ao wrap.           | Timeouts continuam corretos.                                                    |
| AT-10 | Long run               | Executar por 24 h com ciclos repetidos. | Sem travamento, overflow ou energização indevida.                               |

## 14. Plano incremental de desenvolvimento

| Marco                     | Entregas                                                            | Critério para avançar                        |
|---------------------------|---------------------------------------------------------------------|----------------------------------------------|
| M0 - decisões de hardware | Atuador, buzzer, fonte, pinout, mapa PCF8574 e esquemático mínimo.  | Nenhuma decisão elétrica crítica pendente.   |
| M1 - plataforma segura    | GPIO seguro do atuador, timebase, build, board_config e smoke test. | Reset/power-on nunca abre a trava.           |
| M2 - entrada              | Driver keypad + debounce + eventos.                                 | Cada tecla passa teste repetitivo.           |
| M3 - display              | I2C port, PCF8574, adapter e HD44780.                               | LCD inicia e falha sem travar o sistema.     |
| M4 - saídas               | LED, buzzer e lock_actuator com deadlines.                          | Padrões e timeout validados por instrumento. |
| M5 - serviços             | credential_entry, authentication, display/status/sound.             | Testes unitários passam em host.             |
| M6 - controlador          | Máquina de estados completa.                                        | Todas as transições e casos de uso passam.   |
| M7 - integração/fault     | Falhas injetadas, resets, NACK e long run.                          | Matriz AT-01 a AT-10 aprovada.               |
| M8 - release V1           | Documentação, tag, relatório de teste e backlog V2.                 | Definition of Done atendida.                 |

### 14.1 Ordem recomendada das primeiras tarefas

25. Fechar tipo de atuador e circuito de potência antes de escrever a API final do driver.
26. Desenhar tabela de estados/eventos e revisar as invariantes do atuador.
27. Criar esqueleto de pastas e build sem implementar generalidade adicional.
28. Implementar timebase de 1 ms e testar rollover no host.
29. Implementar GPIO seguro do atuador e validar power-on/reset com instrumento.
30. Implementar keypad isolado e medir debounce real.
31. Implementar a cadeia I2C -> PCF8574 -> adapter -> HD44780 em etapas testáveis.
32. Somente então integrar credential\_entry, authentication e lock\_controller.



## 15. Definição de pronto e critérios de release

### 15.1 Definition of Done por módulo

- Contrato público documentado: parâmetros, unidades, retornos, condições e side effects.
- Sem warnings relevantes de compilação; política de warnings definida.
- Dados privados encapsulados; ausência de globals públicos mutáveis.
- Testes correspondentes implementados e aprovados.
- Falhas e parâmetros inválidos tratados conforme contrato.
- Sem delays longos bloqueantes.
- Integração demonstrada no hardware quando o módulo acessa periférico.
- Documentação de pinout/configuração atualizada quando aplicável.

### 15.2 Critérios de release V1

- FR-001 a FR-020 atendidos ou com desvio formalmente aprovado.
- NFR-001 a NFR-016 revisados e evidenciados.
- AT-01 a AT-10 aprovados e registrados.
- Nenhuma falha conhecida capaz de energizar o atuador indefinidamente.
- Long run de 24 h concluído sem travamento.
- Código e documento correspondem à mesma tag de release.
- README permite compilar, gravar e executar o teste básico em máquina limpa.
- Limitações de segurança da V1 documentadas de forma explícita.

## 16. Riscos, decisões pendentes e próximos passos

### 16.1 Registro de riscos

| ID   | Risco                                           | Impacto | Prob.       | Mitigação                                                                   |
|------|-------------------------------------------------|---------|-------------|-----------------------------------------------------------------------------|
| R-01 | Atuador escolhido sem dimensionamento elétrico. | Alto    | Média       | Fechar part number, corrente e estágio antes da integração.                 |
| R-02 | Ruído da carga reinicia ou corrompe o MCU.      | Alto    | Média       | Separação de alimentação, flyback, desacoplamento e teste com osciloscópio. |
| R-03 | LCD backpack possui mapeamento diferente.       | Médio   | Alta        | Levantar mapa por inspeção/medição e centralizar em board_config.           |
| R-04 | Teclado apresenta ghosting/múltiplas teclas.    | Médio   | Média       | Definir política: uma tecla por vez; detectar ambiguidade e ignorar.        |
| R-05 | I2C bloqueia quando periférico ausente.         | Alto    | Média       | Timeout finito e propagação de status.                                      |
| R-06 | Abstrações excessivas atrasam a V1.             | Médio   | Alta        | Exigir requisito/teste para cada interface nova.                            |
| R-07 | PIN exposto no repositório/log.                 | Médio   | Média       | Config separada, sem logs e credencial de demonstração não reutilizada.     |
| R-08 | Reset durante abertura deixa carga ativa.       | Alto    | Baixa/Média | Pull-down físico, init seguro e teste AT-07.                                |
| R-09 | Busy-wait degrada resposta do teclado/atuador.  | Médio   | Média       | Política de não bloqueio e revisão estática.                                |

| ID   | Risco                                           | Impacto | Prob. | Mitigação                                                |
|------|-------------------------------------------------|---------|-------|----------------------------------------------------------|
| R-10 | Sem sensor, estado mecânico diverge do comando. | Alto    | Média | Limitação explícita; planejar sensor de posição para V2. |

## 16.2 Decisões arquiteturais registradas

| ADR     | Decisão                                           | Consequência                                                 |
|---------|---------------------------------------------------|--------------------------------------------------------------|
| ADR-001 | Superloop cooperativo, sem RTOS na V1.            | Menor complexidade; exige atualização não bloqueante.        |
| ADR-002 | Teclado direto em GPIO; PCF8574 dedicado ao LCD.  | Mais pinos, porém drivers independentes e depuração simples. |
| ADR-003 | Autenticação como serviço de domínio, não driver. | Pode ser testada em host e não depende de hardware.          |
| ADR-004 | HD44780 separado do adapter PCF8574.              | Permite trocar o meio físico sem reescrever comandos do LCD. |
| ADR-005 | Atuador com API temporizada e ForceSafe.          | Reduz risco de acionamento indefinido.                       |
| ADR-006 | Sem alocação dinâmica.                            | Uso de memória previsível e menor risco de fragmentação.     |
| ADR-007 | PIN fixo e quatro dígitos.                        | Escopo controlado; não adequado a produto final.             |

## 16.3 Decisões pendentes

| ID   | Pergunta                                                           | Dono              | Prazo recomendado |
|------|--------------------------------------------------------------------|-------------------|-------------------|
| D-01 | Qual é o atuador exato e qual seu estado seguro elétrico/mecânico? | Hardware          | Antes do M1       |
| D-02 | Buzzer ativo ou passivo?                                           | Hardware/software | Antes do M4       |
| D-03 | Qual o mapeamento real do backpack PCF8574?                        | Hardware          | Antes do M3       |
| D-04 | Falha do LCD permite operação degradada ou força FAULT?            | Sistema           | Antes do M6       |
| D-05 | Qual padrão visual/sonoro exato para cada estado?                  | UX/sistema        | Antes do M4       |
| D-06 | Qual ferramenta/framework de teste em host será usado?             | Software          | Antes do M5       |
| D-07 | Watchdog entra na V1 ou imediatamente após a V1?                   | Sistema           | Antes do M7       |

## Apêndice A - Tipos e interfaces conceituais

Os exemplos abaixo fixam contratos e responsabilidades; não são uma implementação completa nem obrigação de nomenclatura literal.

```
typedef enum
{
    LOCK_STATE_BOOT = 0,
    LOCK_STATE_LOCKED_IDLE,
    LOCK_STATE_CREDENTIAL_ENTRY,
    LOCK_STATE_AUTHENTICATING,
    LOCK_STATE_ACCESS_GRANTED,
    LOCK_STATE_ACCESS_DENIED,
    LOCK_STATE_LOCKOUT,
    LOCK_STATE_FAULT
} LockState;

typedef enum
{
    KEY_NONE = 0,
    KEY_0, KEY_1, KEY_2, KEY_3, KEY_4,
    KEY_5, KEY_6, KEY_7, KEY_8, KEY_9,
    KEY_CONFIRM,
    KEY_CANCEL
} KeyCode;

typedef enum
{
    AUTH_RESULT_VALID = 0,
    AUTH_RESULT_INVALID,
    AUTH_RESULT_INVALID_LENGTH,
    AUTH_RESULT_ERROR
} AuthResult;
```

### A.1 Regras para APIs

- Nomear por módulo + ação: LockController\_Update, Pcf8574\_Write, Time\_Elapsed.
- Usar unidades no nome do parâmetro: duration\_ms, frequency\_hz.
- Parâmetros de entrada somente leitura usam const.
- Funções que podem falhar retornam status explícito.
- Handles/contextos recebem estado mutável do módulo; evitar singletons globais por padrão.
- Headers públicos não incluem stm32f1xx\_hal.h salvo módulos da plataforma.
- Callbacks e ponteiros de função só entram quando uma substituição concreta ou teste exige.

## Apêndice B - Rastreabilidade requisito -> módulo

| Requisito(s)            | Módulo(s) responsável(is)                    |
|-------------------------|----------------------------------------------|
| FR-001, FR-013, FR-020  | lock_actuator, lock_controller, board_config |
| FR-003                  | keypad_4x4                                   |
| FR-004 a FR-007, FR-016 | credential_entry, display_service            |
| FR-008 a FR-011         | authentication, lock_controller              |
| FR-012                  | lock_controller, lock_actuator               |
| FR-014                  | status_indicator, led                        |
| FR-015                  | sound_service, buzzer                        |
| FR-017                  | hd44780                                      |
| FR-018                  | hd44780_pcf8574_bus                          |

| Requisito(s)     | Módulo(s) responsável(is)             |
|------------------|---------------------------------------|
| FR-019           | stm32_i2c_port, pcf8574               |
| NFR-005          | time_service e testes host            |
| NFR-007, NFR-008 | separação de camadas e mocks/fakes    |
| NFR-010          | todos os módulos; revisão de memória  |
| NFR-013          | estrutura de pastas e processo CubeMX |

## Apêndice C - Checklist de revisão arquitetural

- ☐ O tipo de atuador e o estado seguro foram definidos por requisito, não por conveniência de código?
- ☐ Existe alguma chamada de HAL fora de Platform/Drivers? Se sim, há justificativa?
- ☐ A aplicação consegue ser compilada/testada sem headers STM32?
- ☐ Cada timeout possui unidade, valor configurável e teste de rollover?
- ☐ Algum módulo conhece uma regra que pertence a outra camada?
- ☐ Há buffer sem limite explícito ou operação de string sem comprimento?
- ☐ Há delay bloqueante maior que o estritamente necessário ao protocolo elétrico?
- ☐ Uma falha do LCD, buzzer ou LED pode impedir o timeout do atuador?
- ☐ Reset e boot mantêm o estágio de potência em estado seguro?
- ☐ O PIN aparece em display, log, dump ou documentação pública?
- ☐ O código gerado pode ser regenerado sem apagar a aplicação?
- ☐ Todos os requisitos possuem teste ou evidência planejada?
- ☐ As abstrações existentes correspondem a variações reais ou testabilidade concreta?
- ☐ O sistema distingue estado comandado do atuador de estado mecânico confirmado?

### Baseline de início

O projeto deve iniciar pelo fechamento das decisões elétricas D-01 a D-03 e pela máquina de estados. Criar drivers antes de resolver essas entradas tende a cristalizar interfaces erradas e aumentar retrabalho.

*Fim do documento - Arquitetura da Fechadura Eletrônica V1*